

Trabajo Practico integrador

Materia:

Gestión del desarrollo de software

Profesor:

Fernández Carbonell, Cesar Augusto

Regional:

UTN Mar del Plata

Integrantes:

Tomas Stutz, Kevin Sánchez , Tomas Sollberger

1.1 - Descripción General

Nombre del proyecto:

Portal web CAAN (Centro de ayuda al animal de Necochea)

Institución seleccionada:

Centro de Ayuda al Animal de Necochea (CAAN), una organización sin fines de lucro enfocada en el rescate, cuidado y adopción responsable de perros callejeros o en situación de riesgo en la ciudad de Necochea.

Problemática identificada:

Actualmente, la información vital de la institución se encuentra dispersa a través de distintas redes sociales, y resulta inaccesible en momentos críticos. Esto genera confusión en las personas cuando están frente a situaciones de emergencia (como encontrar animales en situación de calle), ya que desconocen a qué canal de comunicación acudir, qué números marcar o qué protocolos seguir.

Solución propuesta:

El desarrollo de una plataforma web integral autogestionable que centralice la información institucional. Contará con una sección pública para visualizar el catálogo de perros disponibles (con detalles de salud, tamaño y sexo), un apartado de noticias/eventos para difundir jornadas de castración y campañas, un área de información sobre como contribuir y un panel de administración robusto y seguro para que el personal del CAAN gestione la información (CRUD de animales y noticias) de manera simple y dinámica. Con esto se solucionaría su mayor debilidad que es la información dispersa por todos lados y se conseguiría un lugar central en donde se encuentra toda su información.

1.2- Justificación

Qué necesidad busca resolver el proyecto:

Centralizar y digitalizar la gestión de datos del CAAN. Permite agilizar la búsqueda de hogares para los animales, automatizar la difusión de campañas y simplificar la recaudación de fondos.

Cuál es su aporte para la institución:

Optimiza el tiempo de los voluntarios al automatizar la visualización de perros y reducir las consultas por canales informales. Además, profesionaliza la imagen digital de la organización, facilitando la captación de nuevos socios y donantes.

Qué beneficios podría generar para los usuarios:

permite a la comunidad conocer animales que se encuentran en adopción, saber requisitos para adoptar estos, informarse sobre el maltrato animal y los pasos a seguir para poder denunciar un maltrato, poder ver y estar al tanto de los eventos que realiza el CAAN, y mayor información sobre cómo poder ayudarlos.

2- Misión y visión

2.1 - ¿Para qué existe este proyecto?

Ofrecer al Centro de Ayuda al Animal de Necochea (CAAN) una plataforma digital centralizada y eficiente que conecte ágilmente a la comunidad con los animales en adopción, también con el maltrato animal y cómo actuar. Y cuidados sobre nuestros animales.

2.2- ¿Qué se espera lograr a largo plazo?

Convertir al portal web en el principal centro digital de información y referencia para el bienestar animal en Necochea. A largo plazo, se espera que la plataforma funcione como la guía definitiva para la comunidad, donde los ciudadanos acudan de forma autónoma para descubrir a los animales en adopción, conocer los métodos exactos para enviar donaciones, e informarse sobre los pasos correctos a seguir para realizar denuncias o contactar a la red de rescatistas.

3- Objetivos del Proyecto

3.1 - Objetivo General

Desarrollar e implementar un portal web centralizado para el Centro de Ayuda al Animal de Necochea (CAAN) con el propósito de unificar su comunicación oficial, brindando a la comunidad una plataforma de acceso rápido e intuitivo para consultar el catálogo de animales en adopción, guías de donación y protocolos de acción ciudadana (Denuncias o Preguntas Frecuentes).

3.2 Objetivos Específicos

Definir entre tres (3) y cinco (5) objetivos específicos que contribuyan al cumplimiento del objetivo general.

Objetivo 1: Desarrollar un panel de control centralizado que permita al personal de la institución gestionar el registro, actualización y seguimiento de los animales ingresados de manera autónoma.

Objetivo 2: Implementar un catálogo público interactivo que facilite a los usuarios la búsqueda y visualización de animales en adopción mediante filtros de categorización intuitivos.

Objetivo 3: Establecer un sistema de control de acceso y autenticación para garantizar que la modificación de la información institucional sea realizada exclusivamente por usuarios autorizados.

Objetivo 4: Proveer un módulo de gestión de contenido simplificado para que el staff de la protectora pueda redactar, diseñar y publicar noticias o campañas sin necesidad de conocimientos técnicos previos.

Objetivo 5: Estructurar secciones informativas claras y accesibles que detallen los canales de contacto oficiales, los métodos vigentes para realizar donaciones y los protocolos de acción ciudadana ante emergencias.

4. Identificación de Stakeholders

Nombre o rol	Interés respecto al proyecto	Nivel de influencia
Comisión Directiva del CAAN	Administrar el catálogo de animales de manera ágil, difundir campañas institucionales y transparentar la recepción de donaciones.	Alto
Voluntarios del CAAN	Reducir el tiempo de respuesta ante consultas repetitivas en redes sociales sobre disponibilidad de adopciones y canales de ayuda.	Medio
Adoptantes y Ciudadanos	Acceder rápidamente a la información de contacto ante emergencias, visualizar el catálogo de animales y conocer los pasos para adoptar.	Medio
Donantes y Socios	Disponer de instrucciones claras y centralizadas para aportar fondos o insumos al refugio de manera segura.	Bajo
Tomás Sollberger (Nexo Institucional / Desarrollo Adopciones)	Garantizar que el sistema resuelva la barrera comunicacional que originó el proyecto, gestionando los requerimientos directos de la protectora y aportando al desarrollo del módulo de adopciones.	Alto
Kevin Sánchez (Líder Técnico)	Asegurar la robustez, seguridad	Alto

Backend)	y correcto funcionamiento de la arquitectura del servidor y bases de datos, brindando soporte técnico y resolución de dudas al equipo.	
Tomás Stutz (Líder UI/UX y Frontend)	Diseñar y desarrollar una interfaz gráfica intuitiva, responsive y accesible, asegurando una experiencia de usuario fluida desde cualquier dispositivo.	Alto
Profesor de la Cátedra (Cesar A. Fernández C.)	Evaluar la correcta aplicación de los conceptos teóricos y metodológicos de gestión de software durante todo el ciclo de vida del proyecto.	Alto

5- alcances del Proyecto

5.1 - Funcionalidades Incluidas:

- **Página de Inicio (Home):** Presentación institucional del CAAN, accesos rápidos a las secciones principales y pie de página con enlaces a canales de contacto oficiales.
- **Catálogo Público de Adopciones:** Visualización de animales disponibles mediante un listado filtrable. Incluye fichas individuales detalladas con fotografías, edad, sexo, tamaño, descripción de personalidad y estado de salud (vacunación, castración, desparasitación).
- **Módulo Informativo de Donaciones y Socios:** Espacio con instrucciones precisas sobre cómo realizar aportes económicos (datos bancarios), donaciones físicas (alimento, abrigo, materiales) y los pasos a seguir para asociarse a la institución.
- **Sección de Noticias y Campañas:** Área pública para la difusión de novedades, eventos, campañas de castración y comunicados oficiales del refugio.
- **Sección de Consultas Frecuentes (FAQ):** Respuestas centralizadas a las dudas más comunes sobre los procesos de adopción, donación, protocolos de acción ciudadana y ubicación geográfica.
- **Panel de Administración (Dashboard Interno):**
 - **Gestión de Acceso:** Sistema de autenticación seguro (Inicio y cierre de sesión) exclusivo para el personal autorizado del refugio.

- **Gestión de Catálogo (Animales):** Módulo para registrar (alta), editar, visualizar y dar de baja fichas de animales, incluyendo la carga y administración de sus fotografías.
- **Gestión de Contenido (Noticias):** Módulo con editor de texto integrado para redactar, maquetar, publicar, editar y eliminar artículos o campañas informativas.

5.2 Funcionalidades Excluidas

- **Pasarela de pago integrada:** No se realizará el procesamiento de tarjetas o pagos en tiempo real dentro del sitio; la plataforma se limitará a presentar links externos o datos bancarios para transferencias.
- **Recepción de denuncias formales:** El sistema no contará con formularios para asentar denuncias legales por maltrato animal; su función será puramente informativa, indicando a los ciudadanos los pasos y autoridades correspondientes.
- **Chat interno en tiempo real:** No se proveerá una mensajería directa tipo chat web; toda comunicación directa entre los adoptantes y la institución se derivará a WhatsApp oficiales de los representantes.
- **Aplicación móvil nativa:** El desarrollo será exclusivamente un sistema web responsivo (Desktop y Mobile), sin empaquetado para tiendas de aplicaciones como Android o iOS.

6- Restricciones del Proyecto

- **Tiempo disponible:** El desarrollo y la entrega deben enmarcarse estrictamente dentro del calendario académico del semestre en curso, aunque el proyecto cuenta con un grado de avance óptimo para cumplir los plazos.
- **Recursos humanos:** El equipo está compuesto exclusivamente por tres estudiantes (Kevin Sánchez, Tomás Sollberger, Tomás Stutz), quienes deben cubrir simultáneamente los roles de gestión, diseño de interfaz y programación.
- **Infraestructura disponible (Presupuesto \$0):** Al no contar con financiamiento, el sistema está restringido a utilizar únicamente las capas gratuitas (free-tiers) de servicios en la nube para bases de datos (MongoDB Atlas) y almacenamiento multimedia (Cloudinary).
- **Tecnologías requeridas:** Existe la obligatoriedad de utilizar la pila tecnológica definida (React para Frontend, Node.js/Express para Backend y MongoDB), lo que representó un desafío inicial en la integración de estas herramientas.
- **Conocimientos del equipo:** La principal restricción técnica actual radica en la puesta en producción del sistema. Existe una curva de aprendizaje importante respecto a cómo

configurar el despliegue del software, la gestión de servidores gratuitos, la vinculación de dominios y la migración del código desde los repositorios de Git hacia un entorno público y funcional.

7. Modelo de Desarrollo

Para este proyecto se selecciona el **Modelo de Desarrollo Ágil (Scrum)**

Justificación y razones por las cuales resulta adecuado para el proyecto:

- **Flexibilidad frente a imprevistos:** Al ser un equipo reducido con recursos limitados, el desarrollo incremental nos permite estructurar entregas de valor en cada iteración (Sprints de 2 semanas) y adaptarnos rápidamente si una tecnología (como el almacenamiento multi-cuenta) requiere ajustes.
- **Mitigación del riesgo de integración:** Al desacoplar el Frontend y Backend, un ciclo incremental permite ir construyendo endpoints de la API y consumirlos de inmediato desde React, detectando errores de comunicación tempranamente en lugar de esperar al final del proyecto.
- **Foco en el MVP (Mínimo Producto Viable):** Permite priorizar la funcionalidad core (el catálogo de animales y el panel admin) y dejar para las últimas iteraciones los aspectos estáticos e informativos (FAQ, Donaciones), asegurando que el corazón del sistema esté completamente operativo a la fecha de entrega.
- **Validación continua con el cliente:** Al trabajar mediante iteraciones cortas, el modelo permite presentar versiones funcionales del sistema (como el módulo de adopciones) a la institución de manera periódica, garantizando que el producto se mantenga alineado con la visión y necesidades de la protectora antes de invertir tiempo en otras funcionalidades.

8. Aplicación de Scrum

8.1 Roles

- **Product Owner: Sollberger, Tomás.** Actúa como el nexo principal con el cliente (CAAN) para relevar necesidades, validar el alcance y priorizar el desarrollo. A nivel técnico, participa activamente en el desarrollo full-stack, liderando la configuración inicial de la base de datos en MongoDB y la programación de módulos clave como el CRUD de animales.
- **Scrum Master: Sánchez, Kevin.** Encargado de facilitar el proceso ágil y resolver bloqueos de integración. Además de su rol de gestión, trabaja de forma transversal en el código, desarrollando modelos de datos, arquitectura backend y colaborando en la construcción de vistas en el frontend.

- **Equipo de Desarrollo:** **Sánchez Kevin, Sollberger Tomás, Stutz Tomás Bautista.** Equipo de trabajo multidisciplinario y colaborativo. Si bien la dinámica permite el apoyo cruzado, **Tomás Stutz** asume la responsabilidad principal sobre el diseño UI/UX, la mayor parte del desarrollo del frontend y la elaboración de la documentación del proyecto, mientras que el resto del equipo se enfoca en la lógica de negocio, bases de datos e integración de la API.

8.2 Product Backlog Inicial

- **PB-01:** Configuración del entorno de base de datos MongoDB Atlas y conexión del Backend. **(Prioridad: Alta)**
- **PB-02:** Implementación del sistema de autenticación seguro (JWT dual con cookies httpOnly y endpoint de refresh). **(Prioridad: Alta)**
- **PB-03:** Desarrollo del modelo de datos y endpoints CRUD de Animales (Backend). **(Prioridad: Alta)**
- **PB-04:** Desarrollo del panel de administración del catálogo de animales en Frontend (React). **(Prioridad: Alta)**
- **PB-05:** Desarrollo de la interfaz pública del Catálogo de Animales con filtros por tamaño y sexo (Frontend). **(Prioridad: Alta)**
- **PB-06:** Configuración de la integración con Cloudinary (subida de archivos multimedia). **(Prioridad: Media)**
- **PB-07:** Desarrollo del modelo de datos y endpoints CRUD de Noticias con soporte de texto BlockNote (Backend). **(Prioridad: Media)**
- **PB-08:** Desarrollo del editor y gestor de noticias en el panel de administración (Frontend). **(Prioridad: Media)**
- **PB-09:** Implementación de la sección informativa de Donaciones, Socios y página institucional (Home, FAQ). **(Prioridad: Baja)**
- **PB-10:** Refactorización de código: modularización de API Axios en Frontend y Service Layer en Backend. **(Prioridad: Baja)**

9. Historias de Usuario

- **HU-01:** Como adoptante, quiero **visualizar el catálogo de perros del CAAN con sus fotos y datos de salud**, para **poder evaluar de forma informada cuál de ellos se adaptaría mejor a mi hogar**.

- **HU-02:** Como adoptante, quiero filtrar los animales por tamaño (pequeño, mediano, grande) y por sexo (macho, hembra), para buscar rápidamente perros que cumplan con los requisitos de espacio de mi vivienda.
- **HU-03:** Como donante, quiero acceder a los datos de transferencia bancaria e información de insumos necesarios en la web, para realizar aportes físicos o monetarios de forma rápida y confiable.
- **HU-04:** Como vecino de Necochea, quiero leer las noticias y campañas en la plataforma, para enterarme sobre las próximas jornadas de castración gratuita y eventos benéficos.
- **HU-05:** Como administrador del refugio, quiero iniciar sesión mediante usuario y contraseña de forma segura, para acceder al panel de gestión y evitar accesos no autorizados a los datos.
- **HU-06:** Como administrador del refugio, quiero registrar un nuevo animal ingresando su nombre, edad, tamaño, fotos y estado de salud, para que se muestre de inmediato en el catálogo público.
- **HU-07:** Como administrador del refugio, quiero modificar el estado de un perro a "Adoptado" o editar su información de salud, para mantener el catálogo actualizado y evitar que los usuarios consulten por animales que ya encontraron un hogar.
- **HU-08:** Como administrador del refugio, quiero eliminar de forma permanente la ficha de un animal, para limpiar el catálogo de registros obsoletos o erróneos.
- **HU-09:** Como administrador del refugio, quiero redactar noticias utilizando un editor de texto enriquecido por bloques, para publicar anuncios institucionales atractivos y con formato sin requerir conocimientos técnicos de programación.
- **HU-10:** Como administrador del refugio, quiero que el sistema gestione de manera automática la subida de imágenes a cuentas secundarias si la cuenta principal está saturada, para poder seguir actualizando la web sin interrupciones operativas.

10. Requerimientos

10.1 Requerimientos Funcionales

- **RF1 (Autenticación):** El sistema debe validar las credenciales de los administradores y gestionar la sesión utilizando cookies httpOnly seguras con tokens JWT.
- **RF2 (Registro de Animales):** El sistema debe permitir al administrador dar de alta un animal registrando nombre, especie, raza, fecha de nacimiento, sexo, tamaño, estado (Disponible/Adoptado), salud (vacunado, castrado, desparasitado, condiciones especiales), descripción e imágenes.

- **RF3 (Edición de Animales):** El sistema debe permitir al administrador modificar los datos de cualquier animal registrado mediante formularios de edición.
- **RF4 (Eliminación de Animales):** El sistema debe permitir al administrador borrar registros de animales de la base de datos.
- **RF5 (Consulta de Catálogo):** El sistema debe mostrar al público general la lista de animales disponibles para adopción, con paginación y filtros dinámicos.
- **RF6 (Creación de Noticias):** El sistema debe permitir al administrador crear noticias con título, slug descriptivo único, contenido dinámico en formato JSON, imagen de portada y categoría.
- **RF7 (Edición de Noticias):** El sistema debe permitir modificar el contenido y estado de publicación (borrador/publicado) de las noticias.
- **RF8 (Eliminación de Noticias):** El sistema debe permitir al administrador borrar noticias de la plataforma.
- **RF9 (Subida Multimedia):** El sistema debe procesar archivos de imagen a través del backend, subirlos a Cloudinary y almacenar la URL resultante en la base de datos.
- **RF10 (Visualización Pública de Contenido):** El sistema debe renderizar las vistas informativas de Home, Donaciones, Preguntas Frecuentes y Noticias de forma libre (sin necesidad de loguearse).

10.2 Requerimientos No Funcionales

- **RNF1 (Seguridad):** Los tokens JWT de autenticación no deben ser accesibles a través de script del lado del cliente (configurados como cookies httpOnly y Secure) para mitigar ataques de Cross-Site Scripting (XSS).
- **RNF2 (Confiabilidad y Fallback):** El módulo de subida a la nube debe implementar una cadena de responsabilidad (*Chain of Responsibility*) con hasta tres cuentas de Cloudinary para evitar interrupciones por límites de cuota gratuita.
- **RNF3 (Usabilidad y Responsividad):** La interfaz de usuario debe desarrollarse bajo la metodología *Mobile-First* utilizando Tailwind CSS, garantizando una correcta visualización en pantallas móviles y de escritorio.
- **RNF4 (Robustez y Validación):** Todos los datos de entrada en los endpoints de la API (Backend) deben ser validados sintáctica y semánticamente mediante esquemas de Zod antes de ser procesados o guardados en MongoDB.
- **RNF5 (Mantenibilidad y Arquitectura):** El backend debe estructurarse bajo una arquitectura de capas bien definidas (Rutas, Middlewares, Controladores, Modelos y Schemas), asegurando el principio de responsabilidad única (SRP).

11. Priorización de Requerimientos

- **Prioridad Alta:** RF1, RF2, RF3, RF4, RF5, RNF1, RNF4. *Justificación:* Constituyen el núcleo operativo (*Minimum Viable Product*) de la aplicación. Sin un sistema de autenticación seguro, un catálogo público accesible y la capacidad completa de gestionar (dar de alta, editar y eliminar) las fichas de los animales, la plataforma no cumple con el propósito mínimo solicitado por el refugio. Asimismo, las validaciones estrictas en el backend garantizan la integridad y consistencia de los datos almacenados.
- **Prioridad Media:** RF6, RF7, RF8, RF9, RNF2, RNF3, RNF5. *Justificación:* El módulo de noticias, la carga multimedia y el diseño responsivo (*Mobile-First*) enriquecen significativamente la experiencia de usuario y optimizan la difusión de campañas en la comunidad. El mecanismo de *fallback* de almacenamiento y la arquitectura en capas aseguran la confiabilidad, robustez y mantenibilidad del software a mediano plazo.
- **Prioridad Baja:** RF10. *Justificación:* La renderización de las secciones informativas estáticas (como la página de inicio, los datos para donaciones y el apartado de preguntas frecuentes), aunque son fundamentales para la comunicación institucional, representan contenido estático cuya implementación puede postergarse hacia las iteraciones finales del ciclo lectivo sin comprometer la lógica de negocio ni la arquitectura del sistema.

12. Gestión de Riesgos

Riesgo 1 (R1) - Saturación del almacenamiento multimedia

- **Descripción:** Al utilizar la capa gratuita de Cloudinary, el volumen de imágenes en alta resolución subidas por el refugio podría superar el límite mensual de almacenamiento o ancho de banda, bloqueando la carga de nuevos animales.
- **Tipo de riesgo:** Técnico.

Riesgo 2 (R2) - Indisponibilidad o deserción de un miembro del equipo

- **Descripción:** Al ser un equipo reducido de solo tres integrantes, la enfermedad, problemas personales o sobrecarga horaria de alguno de los desarrolladores puede comprometer seriamente los plazos de entrega del software.
- **Tipo de riesgo:** Humano.

Riesgo 3 (R3) - Retrasos en el cronograma por la complejidad del despliegue

- **Descripción:** La falta de experiencia previa del equipo en la puesta en producción (configuración de servidores, variables de entorno inter-dominio y despliegue del monorepo) podría dilatar los tiempos de entrega en la fase final del ciclo lectivo.
- **Tipo de riesgo:** Gestión.

Riesgo 4 (R4) - Acoplamiento excesivo en la lógica del servidor

- **Descripción:** Mezclar responsabilidades dentro de los controladores del backend (como el procesamiento de texto enriquecido, el formateo de datos y la subida a la nube) puede generar un código difícil de mantener y testear de forma aislada.
- **Tipo de riesgo:** Técnico.

Riesgo 5 (R5) - Filtración de información sensible del sistema

- **Descripción:** La ausencia de un manejador de errores global y homogéneo en la API podría provocar que ante un fallo del servidor se expongan trazas internas de la base de datos (MongoDB) hacia el cliente, vulnerando la seguridad.
- **Tipo de riesgo:** Técnico.

Riesgo 6 (R6) - Abandono del sistema por parte del personal del refugio

- **Descripción:** Que los voluntarios o la Comisión Directiva del CAAN encuentren complejo el panel de administración o no adopten el hábito de actualizarlo, provocando que la plataforma quede obsoleta y desinforme a la comunidad.
- **Tipo de riesgo:** Negocio.

13. Matriz de Riesgos

Íd	Descripción del Riesgo	Categoría	Probabilidad de ocurrencia	Impacto esperado	Nivel de Riesgo
R1	Saturación de cuota en almacenamiento de imágenes (Cloudinary).	Técnico	Media	Medio	Medio
R2	Indisponibilidad o deserción de un miembro	Humano	Baja	Alto	Medio

	del equipo.				
R3	Retrasos en el cronograma por la complejidad del despliegue (<i>deploy</i>).	Gestión	Alta	Alto	Alto
R4	Acoplamiento excesivo en la lógica del servidor (Controladores No-SRP).	Técnico	Media	Medio	Medio
R5	Filtración de información sensible por errores del servidor.	Técnico	Media	Alto	Alto
R6	Abandono o falta de adopción del sistema por parte del refugio.	Negocio	Baja	Alto	Medio

14. Estrategias de Mitigación

- **Mitigación para R1 (Saturación de cuota en Cloudinary):** Implementar en el backend un módulo de subida multimedia que aplique el patrón de diseño *Chain of Responsibility* (Cadena de Responsabilidad). Se configurarán hasta tres cuentas gratuitas de Cloudinary en las variables de entorno; si una cuenta satura su cuota y retorna un error, el sistema derivará de forma automática y transparente la carga de imágenes hacia la siguiente cuenta de la cadena.
- **Mitigación para R2 (Indisponibilidad o deserción de un miembro del equipo):** Mantener un repositorio en GitHub con un flujo de trabajo claro, ramas ordenadas y documentación interna al día en cada sección del código. Al estar el código bien documentado y estructurado, cualquier integrante podrá asumir temporalmente las tareas pendientes de otro compañero ante una eventualidad o ausencia, minimizando el impacto en el avance del grupo.
- **Mitigación para R3 (Retrasos en el cronograma por la complejidad del despliegue):** Adelantar las tareas de investigación y configuración del entorno de producción. Se realizarán pruebas de despliegue preliminares (*deploys* tempranos) a mitad de la cursada utilizando servidores y plataformas en la nube gratuitos, en lugar de esperar a la semana de entrega. Esto permitirá identificar errores de variables de entorno y conexión inter-dominio con suficiente anticipación.
- **Mitigación para R4 (Acoplamiento excesivo en la lógica del servidor):** Introducir una capa de servicios (*Service Layer*). Se extraerá la lógica compleja de los controladores hacia archivos de servicios dedicados (por ejemplo, `src/services/news.service.js` o `src/services/animals.service.js`). Estos archivos se encargarán de procesar los datos, parsear

formatos JSON de BlockNote y coordinar con Cloudinary, dejando al controlador con la única función limpia de recibir la petición y responder HTTP.

- **Mitigación para R5 (Filtración de información sensible por errores del servidor):**
Desarrollar un Middleware de Manejo de Errores Centralizado en Express (src/middlewares/errorHandler.js). Todos los bloques catch de los controladores delegarán los fallos a través de la función next(error), de modo que el middleware capture el error en un solo punto, genere registros internos (*logs*) y responda al cliente un JSON con un mensaje genérico y seguro, ocultando trazas de MongoDB.
- **Mitigación para R6 (Abandono o falta de adopción del sistema por parte del refugio):**
Diseñar una interfaz en el panel de administración sumamente intuitiva, simple y libre de tecnicismos, adaptada al nivel del staff del refugio. Además, se coordinará una breve sesión de capacitación técnica con los voluntarios una vez finalizado el desarrollo y se les entregará un manual de usuario simplificado en formato PDF.

15. Calidad del Software

- **Adecuación funcional:** Se garantiza al cubrir de forma estricta las necesidades de comunicación pública y administración interna que requiere el CAAN. La plataforma implementa las funcionalidades core definidas en el alcance (catálogo, noticias e información institucional), mientras que las validaciones rigurosas con Zod en el backend aseguran que no se procesen datos incompletos o erróneos.
- **Usabilidad:** Se asegura mediante una interfaz limpia y responsiva diseñada bajo el enfoque *Mobile-First* con Tailwind CSS, facilitando la navegación de los adoptantes desde celulares. Para el panel administrativo, la integración del editor BlockNote permite al personal de la protectora redactar y maquetar noticias de forma intuitiva mediante bloques visuales, sin requerir conocimientos de programación.
- **Eficiencia:** El rendimiento del sistema se optimiza en el backend mediante el uso de índices estratégicos en la base de datos de MongoDB Atlas para acelerar las búsquedas y filtrados en el catálogo. En el frontend, al tratarse de una arquitectura desacoplada, las peticiones asíncronas con Axios evitan recargas completas de la página, reduciendo los tiempos de respuesta y el consumo de ancho de banda.
- **Fiabilidad:** El software implementa mecanismos de tolerancia a fallos y protección operativa. El backend cuenta con middlewares de límite de peticiones (express-rate-limit) para prevenir sobrecargas en el servidor, mientras que el módulo de subida multimedia aplica una cadena de responsabilidad (*Chain of Responsibility*) con cuentas secundarias en Cloudinary, garantizando la disponibilidad del servicio ante la saturación de cuotas gratuitas.

- **Seguridad:** Se garantiza la confidencialidad e integridad de la plataforma mediante un sistema de autenticación robusto. Los tokens JWT se almacenan exclusivamente en cookies con atributos httpOnly y Secure, blindando el sistema contra ataques de Cross-Site Scripting (XSS). Adicionalmente, las contraseñas de los administradores se almacenan encriptadas en la base de datos mediante el algoritmo de hasheo seguro bcrypt.
- **Mantenibilidad:** La arquitectura del sistema se estructuró bajo el principio de responsabilidad única (SRP). Al separar el backend en capas bien definidas (Rutas, Middlewares, Controladores y una capa de Modelos) y modularizar las llamadas de la API en el frontend, se logra un código altamente desacoplado, fácil de testear de forma aislada y preparado para escalar con nuevas funciones sin romper el comportamiento existente.

16. Estrategia de Testing

1. Pruebas Unitarias (Unit Testing)

- **Objetivo:** Validar de forma aislada e independiente el correcto funcionamiento de la lógica de negocio y las funciones críticas del código, tales como la subida en cadena de Cloudinary, los esquemas de validación sintáctica de Zod y los modelos de la capa intermedia.
- **Momento de aplicación:** En paralelo a la codificación de cada módulo o servicio (durante la fase de desarrollo activa de cada Sprint).

2. Pruebas de Integración (Integration Testing)

- **Objetivo:** Probar la interacción y el correcto flujo de comunicación entre componentes interdependientes del sistema. Específicamente, verificar que los endpoints de la API ejecuten correctamente la secuencia de middlewares establecidos (control de flujo, autenticación JWT, validación de esquemas Zod) antes de impactar en la base de datos de MongoDB.
- **Momento de aplicación:** Al finalizar la construcción de una ruta o un flujo de datos completo en la API, antes de ser consumida por el frontend.

3. Pruebas de Sistema / Extremo a Extremo (E2E Testing)

- **Objetivo:** Verificar el flujo completo del usuario final simulando la interacción en el navegador con la interfaz de React, asegurando que el consumo de la API y la persistencia de datos funcionen en sincronía (por ejemplo, iniciar sesión con éxito, verificar el redireccionamiento al dashboard administrativo, dar de alta un perro con su foto y corroborar que se renderice en el catálogo público).

- **Momento de aplicación:** En las fases de cierre de cada Sprint, una vez que el Frontend y el Backend están integrados, asegurando la estabilidad del incremento.
- **4. Pruebas de Aceptación**
 - **Objetivo:** Corroborar junto a la Comisión Directiva y los voluntarios del CAAN que las interfaces, el catálogo de adopciones y la sección de noticias cumplan con los criterios de aceptación del negocio, garantizando que el sistema sea fácil de usar para ellos y resuelva la problemática inicial de desinformación.
 - **Momento de aplicación:** Al cierre del proyecto, durante la etapa de validación del Mínimo Producto Viable (MVP) previo a la puesta en producción definitiva.

17. Deuda Técnica

¿Cómo podría originarse?

- **Presión de los plazos académicos:** La necesidad de cumplir estrictamente con el calendario de entregas de la cursada podría tentar al equipo a priorizar la velocidad sobre la calidad.
- **Priorización del funcionamiento inmediato:** Postergar la modularización del backend y mezclar la lógica de negocio, el formateo de datos y la gestión multimedia directamente dentro de los controladores con tal de ver el sistema operativo a corto plazo.
- **Ausencia inicial de pruebas automatizadas:** El desarrollo continuo de funcionalidades sin el soporte de tests unitarios o de integración que validen la estabilidad del código ante futuros cambios.

¿Qué consecuencias tendría?

- **Fragilidad del código y aumento de bugs:** Un incremento exponencial en el tiempo requerido para corregir errores sencillos, ya que la modificación de un módulo podría alterar o romper funcionalidades en otras secciones del sistema debido al alto acoplamiento.
- **Fricciones en el trabajo colaborativo:** Aparición constante de conflictos complejos al unir ramas en Git (*merge conflicts*), ralentizando el ritmo de desarrollo de los tres integrantes.
- **Dificultad de escalabilidad:** Resistencia del software a incorporar nuevas características en el futuro y complicaciones para que un desarrollador externo entienda la arquitectura debido a la falta de claridad técnica.

Acciones para evitarla o reducirla:

- **Asignación de tiempo para refactorización:** Reservar un porcentaje estimado del 15% del tiempo en cada Sprint de Scrum exclusivamente para limpiar código, mejorar la estructura y saldar la deuda técnica acumulada.
- **Adopción temprana de patrones arquitectónicos:** Implementar de manera rigurosa la capa de servicios (*Service Layer*) en el backend, la división de módulos de Axios por dominios en el frontend y el patrón *Chain of Responsibility* para los servicios en la nube desde las primeras etapas del desarrollo.
- **Automatización de estándares de calidad:** Configurar herramientas de análisis estático de código (como *ESLint* y *Prettier*) en el entorno de desarrollo para obligar al equipo a mantener estándares homogéneos de formateo, legibilidad y buenas prácticas sintácticas en cada confirmación de código (*commit*).

18 - Conclusión

El desarrollo del proyecto para el Centro de Ayuda al Animal de Necochea (CAAN) ha demostrado que el éxito de un producto de software no depende únicamente de la habilidad para escribir código, sino principalmente de una **gestión planificada, empática y estructurada del desarrollo**.

El valor de este proyecto radica en haber nacido de una problemática comunitaria real: la dificultad de los ciudadanos para acceder a canales de contacto centralizados ante emergencias animales en la vía pública. Abordar esta necesidad nos exigió aplicar conceptos fundamentales de gestión, tales como la delimitación estricta del alcance (diferenciando las secciones informativas de los procesos transaccionales), el control de las restricciones de presupuesto cero y el análisis proactivo de riesgos técnicos. Asimismo, la adopción del marco metodológico Scrum resultó indispensable para coordinar los esfuerzos de un equipo reducido, permitiendo una división clara de roles donde la vinculación con la institución, el diseño de la interfaz de usuario (UI/UX) y la arquitectura del servidor funcionaron en constante sincronía a través de iteraciones incrementales.

Las principales dificultades encontradas no surgieron en la codificación de las funcionalidades básicas, sino en la integración y la arquitectura avanzada del sistema. El equipo debió afrontar una curva de aprendizaje importante para resolver el acoplamiento en el servidor mediante una capa de servicios (*Service Layer*), asegurar el flujo de autenticación ínter-dominio y, fundamentalmente, enfrentar el desafío actual de la **puesta en producción**. Configurar el despliegue de un monorepo, gestionar servidores en la nube bajo capas gratuitas y vincular los dominios web representaron los obstáculos de gestión técnica más complejos del proceso.

Como aprendizaje principal, este trabajo integrador consolida la perspectiva de que la gestión de software es una disciplina holística. Entendimos que las decisiones arquitectónicas (como el uso de esquemas de validación con Zod o el patrón de Cadena de Responsabilidad para mitigar la saturación multimedia) impactan de forma directa en atributos de calidad como la seguridad y la mantenibilidad. En conclusión, la experiencia nos demostró que gestionar, diseñar y programar

deben ir siempre de la mano para transformar una necesidad social en una solución tecnológica funcional, robusta y con impacto real.

19. Presentación del Proyecto

19.1 Estrategia de Estimación Seleccionada

Para el desarrollo del portal web del CAAN, se ha seleccionado una estrategia ágil de estimación basada en **Story Points (Puntos de Historia)**, complementada con las técnicas de **Planning Poker** y **Estimación por Analogía**.

Justificación y adecuación para el proyecto:

- **Coherencia con el marco ágil:** Al haber adoptado Scrum como modelo de desarrollo, estimar el esfuerzo en horas de reloj absolutas suele inducir a desvíos debido a la variabilidad en la experiencia técnica de cada desarrollador y las incertidumbres propias de la integración de software.
- **Medición multidimensional:** Los *Story Points* permiten al equipo establecer una métrica relativa que unifica de forma holística tres factores críticos: la **complejidad técnica** de la funcionalidad, el **esfuerzo físico requerido** para su codificación y el **nivel de riesgo o incertidumbre** asociado (como la configuración inicial de entornos desconocidos).
- **Dinámica colaborativa (Planning Poker):** Mediante el uso de la secuencia de Fibonacci modificada (1, 2, 3, 5, 8, 13), el equipo completo participa activamente en la asignación de puntos. Esto fomenta el debate técnico temprano sobre cómo resolver cada tarea y evita que las estimaciones se vuelvan subjetivas o unilaterales.
- **Uso de Historias Pivote (Analogía):** El método resulta ideal para la estructura de nuestro equipo, ya que nos permite calcular el esfuerzo comparando requerimientos nuevos contra componentes de referencia ya analizados. Por ejemplo, se establece una relación de escala asignando una puntuación baja a una vista estática e informativa como la sección de Preguntas Frecuentes (**2 Story Points**), en contraposición a una funcionalidad compleja con alta incertidumbre de seguridad como la autenticación JWT con cookies httpOnly (**8 Story Points**).

19.2 Descomposición de Funcionalidades

Módulo / Componente	ÍD Tarea	Descripción de la Tarea / Funcionalidad
Capa Transversal	T-1.1	Configuración del entorno de base de datos en MongoDB Atlas y establecimiento de la conexión mediante Mongoose.
	T-1.2	Implementación de validaciones de entrada seguras utilizando esquemas de Zod en los endpoints del servidor.
	T-1.3	Configuración inicial del servidor Express (CORS, parser de cookies, manejo global de errores y variables de entorno).
Autenticación (Auth)	T-2.1	API: Desarrollo de endpoints para Login, Logout y Verificación de Estado de la sesión.
	T-2.2	API: Implementación del middleware de autenticación y la lógica de doble token (<i>Access Token</i> y <i>Refresh Token</i> en cookies httpOnly).
	T-2.3	Frontend: Creación del AuthContext en React y configuración de Axios con interceptores para la renovación automática del token.
	T-2.4	Frontend: Diseño de la interfaz visual de Login y configuración de rutas protegidas para el acceso al panel de administración.
Gestión de Animales	T-3.1	API: Desarrollo del CRUD completo y endpoints para el catálogo de animales en la base de datos.
	T-3.2	API: Configuración de la integración con Cloudinary aplicando el patrón <i>Chain of Responsibility</i> para soporte multi-cuenta.
	T-3.3	Frontend: Desarrollo de la vista pública del Catálogo de Adopciones con filtros dinámicos (sexo, tamaño) y paginación.
	T-3.4	Frontend: Diseño del formulario dinámico de

Módulo / Componente	ÍD Tarea	Descripción de la Tarea / Funcionalidad
		creación y edición de animales en el Panel Admin, incluyendo carga de imágenes.
Blog de Noticias	T-4.1	API: Desarrollo del CRUD completo de noticias (gestión de título, slug descriptivo único, categoría y estado de publicación).
	T-4.2	Frontend: Integración del editor de texto enriquecido <i>BlockNote</i> en el panel administrativo para la maquetación de campañas.
	T-4.3	Frontend: Diseño de la vista pública del listado de noticias y la pantalla de detalle individual basada en el slug.
Vistas Generales	T-5.1	Frontend: Maquetación y diseño responsivo (<i>Mobile-First</i>) de la Página de Inicio (Home), sección informativa de Donaciones/Socios y el módulo de Preguntas Frecuentes (FAQ) interactivas.

19.3 Estimación de Esfuerzo

ÍD Tarea	Funcionalidad / Tarea	Esfuerzo (SP)	Justificación de la Estimación
T- 1.1	Configurar MongoDB Atlas	2 SP	Tarea técnica conocida por el equipo, requiere configuración cloud básica del clúster.
T -1.2	Validaciones Zod	3 SP	Requiere modelar esquemas estrictos de validación en el servidor para cada entidad (animal, noticia, Login).
T- 1.3	Configurar Express	2 SP	Implementación de <i>boilerplate</i> estándar de backend y middlewares de base (CORS, parser).
T-2.1	Endpoints de Auth	3 SP	Lógica de base de datos con

			encriptación bcrypt y endpoints HTTP tradicionales.
T-2.2	Cookies httpOnly & JWT	8 SP	Complejidad Alta: Requiere configuración estricta de cabeceras seguras, control del ciclo de vida de los tokens y middlewares cruzados.
T-2.3	AuthContext & Axios Interceptor	8 SP	Complejidad Alta: Programación de interceptores para capturar errores 401 y realizar renovaciones silenciosas de tokens sin interrumpir la experiencia de usuario.
T-2.4	Frontend Login & Protected Routes	3 SP	Desarrollo de formulario simple y componentes envoltorios (<i>wrappers</i>) para rutas protegidas en React Router.
T-3.1	API CRUD Animales	3 SP	Lógica REST estándar apoyada en controladores y modelos de Mongoose.
T-3.2	Fallback Cloudinary	5 SP	Requiere diseñar el patrón algorítmico <i>Chain of Responsibility</i> para alternar dinámicamente entre credenciales ante saturación de cuota.
T-3.3	Vista Pública Catálogo	5 SP	Lógica de gestión de estados en el cliente para el filtrado dinámico y renderizado responsivo de las tarjetas de animales.
T-3.4	Formulario Admin Perros	8 SP	Complejidad Alta: Manejo de estado complejo en el panel interno y subida asíncrona de múltiples archivos de imagen.
T-4.1	API CRUD Noticias	3 SP	Estructura muy similar al CRUD de animales, lo que reduce drásticamente la incertidumbre técnica.
T-4.2	Editor BlockNote en Admin	8 SP	Integración y compatibilidad del editor de bloques de terceros con el estado de

T-4.3	Listado y Detalle de Noticias	3 SP	formularios en React. Renderizado estructurado en las vistas públicas del árbol de nodos JSON devuelto por BlockNote.
T-5.1	Vistas Generales (Home, Donaciones, FAQ)	3 SP	Historia Pivote: Tarea orientada al desarrollo visual, con baja interactividad lógica y nula incertidumbre.
Total	Esfuerzo Consolidado del Proyecto	66 SP	Estimación global cerrada por el equipo completo a través de la dinámica de Planning Poker.

19.4 Análisis de Factores que Afectan la Estimación

- **Complejidad Técnica:** El mayor factor de variación radica en la arquitectura de seguridad y el flujo de autenticación. La implementación de la lógica transaccional de tokens duales (JWT) almacenados en cookies con atributos httpOnly y Secure presenta una complejidad elevada al cruzar subdominios o puertos diferentes en entornos locales (Backend en puerto 3000 y Frontend en puerto 5173). Las restricciones de políticas de los navegadores modernos (*SameSite*, *CORS*) representan un desafío técnico que suele dilatar los tiempos estimados.
- **Experiencia del Equipo y Curva de Aprendizaje:** Aunque el equipo posee una base sólida en desarrollo web, la adopción de las versiones más recientes de las tecnologías core (como React y Tailwind CSS) introduce cambios sintácticos y nuevas estructuras de configuración. El proceso de adaptación a estas herramientas genera una curva de aprendizaje inicial que podría ralentizar el ritmo de maquetación y desarrollo del frontend durante los primeros sprints.
- **Riesgos Identificados:** Las desviaciones en las estimaciones también están sujetas a la ocurrencia de los riesgos críticos analizados en la primera entrega, fundamentalmente el acoplamiento de lógica en los controladores y las dificultades en la puesta en marcha final. Resolver la configuración de variables de entorno ínter-dominio y el despliegue dinámico del monorepo en servidores en la nube gratuitos constituye una fuente de incertidumbre operativa que puede alterar el cronograma.
- **Dependencias Externas:** El sistema depende directamente de servicios integrados de terceros en sus capas gratuitas, específicamente MongoDB Atlas para la base de datos en la nube y Cloudinary para el almacenamiento multimedia. Cualquier corte temporal en el servicio de sus API, modificaciones en sus políticas de uso o demoras al implementar el algoritmo de conmutación de cuentas (*Chain of Responsibility*) impactará de forma directa en las fases de testing y desarrollo del catálogo.

- **Cambios en los Requerimientos e Integración:** Al tratarse de un desarrollo colaborativo sobre una Aplicación de Página Única (SPA) por parte de tres programadores en paralelo, la frecuencia de las integraciones en el repositorio de GitHub es un factor crítico. La aparición de colisiones de código (*merge conflicts*) complejas o la necesidad de reajustar componentes del panel administrativo para adaptarlos al nivel técnico de los voluntarios del refugio demandarán horas adicionales de refactorización no contempladas en la estimación lineal inicial.

20. Recursos del Proyecto

20.1 Recursos Humanos

Sollberger, Tomás

- **Rol:** *Product Owner / Backend & DevOps Engineer.*
- **Responsabilidades:** Representar las necesidades del refugio CAAN, priorizar el *Product Backlog*, validar los criterios de aceptación de las historias de usuario, configurar la estructura de la base de datos en MongoDB Atlas, establecer el entorno de desarrollo basado en monorepo (*pnpm workspaces*), programar los esquemas transversales de validación con Zod y coordinar la estrategia de despliegue y puesta en producción en la nube.
- **Participación estimada:** 15 horas semanales (equivalente al **33.3%** del esfuerzo total del equipo durante el ciclo de desarrollo).

Sánchez, Kevin

- **Rol:** *Scrum Master / Full-Stack Developer.*
- **Responsabilidades:** Facilitar las ceremonias de la metodología ágil, remover impedimentos del equipo, codificar los controladores del backend (lógica transaccional de animales y noticias), implementar el sistema de autenticación JWT y coordinar las pruebas de integración globales junto con el área de QA Testing.
- **Participación estimada:** 15 horas semanales (equivalente al **33.3%** del esfuerzo total del equipo durante el ciclo de desarrollo).

Stutz, Tomás Bautista

- **Rol:** *Frontend Developer / UI-UX Designer.*

- **Responsabilidades:** Diseñar la arquitectura de información y prototipos visuales en Figma, maquetar la interfaz pública responsiva (*Mobile-First*) y el panel de administración utilizando React y Tailwind CSS, gestionar los estados globales de la aplicación, integrar librerías de terceros (como el editor *BlockNote*) y redactar la documentación y manuales de usuario para el refugio.
- **Participación estimada:** 15 horas semanales (equivalente al **33.3%** del esfuerzo total del equipo durante el ciclo de desarrollo).

20.2 Recursos Tecnológicos

- **lenguajes y Entornos:** JavaScript (ES Modules), Node.js (v20 o superior) como entorno de ejecución en el servidor, y Vite (v6+) como herramienta de construcción y empaquetado del Frontend.
- **Frameworks y Librerías Backend:** Express.js (v5+) para la arquitectura del servidor, Mongoose como ODM para el modelado de datos, jsonwebtoken para el control de sesiones, bcrypt para el hash de contraseñas, Zod para la validación estricta de esquemas y Multer (configurado en memoria) para interceptar la carga de archivos multimedia.
- **Frameworks y Librerías Frontend:** React (v19) para la construcción de la interfaz de usuario, React Router DOM (v7) para el enrutamiento y control de vistas protegidas, Tailwind CSS (v4) para el diseño estilizado responsivo, Axios como cliente HTTP para el consumo de la API, React Hook Form para la gestión eficiente de formularios y BlockNote como editor de texto enriquecido por bloques.
- **Herramientas de Diseño:** Figma, utilizado para el prototipado de alta fidelidad, la maqueta de la interfaz responsiva (*wireframes*) y la definición de la guía de estilos visuales.
- **Control de Versiones y Gestión:** Git como sistema de control de versiones local, GitHub para el alojamiento de repositorios remotos colaborativos y *pnpm workspaces* para la administración eficiente de dependencias dentro de la arquitectura de monorepo.
- **Servicios de Alojamiento Cloud:** MongoDB Atlas (Database as a Service) para la persistencia de datos en la nube y Cloudinary como servicio de almacenamiento distribuido de archivos multimedia (imágenes del catálogo y noticias).

20.3 Recursos de Infraestructura

- **Estaciones de Trabajo Locales (Entorno de Desarrollo):** Computadoras personales de los integrantes del equipo con sistemas operativos Windows/Linux, configuradas con variables

de entorno locales y servidores de escucha en los puertos 3000 (servidor Express) y 5173 (servidor de desarrollo de Vite).

- **Bases de Datos Virtuales:** Instancia de base de datos no relacional alojada en la nube mediante un clúster *Sandbox* gratuito en MongoDB Atlas, con réplicas automatizadas para asegurar la disponibilidad durante las pruebas.
- **Servicio de Almacenamiento de Medios:** Cuentas virtuales en la nube de Cloudinary que proveen hasta 25 GB de almacenamiento optimizado y transformación de imágenes en tiempo real a través de su CDN gratuita.
- **Hosting de Producción (Entorno de Puesta en Producción):** Plataformas e infraestructura en la nube optimizadas para el despliegue del monorepo: **Vercel** para el alojamiento y distribución global de la aplicación estática del Frontend (React), y **Render** (o servicios cloud equivalentes) para el despliegue continuo del servicio web del Backend (Node.js/Express).

21. Costos del Proyecto

21.1 Costos Humanos

Realizar una estimación del costo asociado al trabajo del equipo de desarrollo, asumiendo valores de mercado en pesos argentinos (ARS):

Kevin S. (PM / Full Stack): 15 hs/semana * 6 semanas = 90 hs. Valor hora: \$4.500 ARS. Costo: \$405.000 ARS.

Tomás S. (DevOps / DBA): 15 hs/semana * 6 semanas = 90 hs. Valor hora: \$4.500 ARS. Costo: \$405.000 ARS.

Tomás B. (Frontend / UX): 15 hs/semana * 6 semanas = 90 hs. Valor hora: \$4.000 ARS. Costo: \$360.000 ARS.

Costo Humano Total: \$1.170.000 ARS (durante la duración del desarrollo de 6 semanas).

21.2 Costos Tecnológicos

MongoDB Atlas: TIER M0 Sandbox (Gratuito - 512 MB de almacenamiento) -> \$0 ARS.

Cloudinary: Plan Gratuito (Suficiente para el volumen inicial de fotos del CAAN) -> \$0 ARS.

Hosting (Vercel/Render): Planes Hobby Gratuitos -> \$0 ARS.

Licencias de Software (VS Code, Git, pnpm): Código Abierto / Gratuitos -> \$0 ARS.

Dominio Web Personalizado: Compra de un dominio `.org.ar` para formalizar la institución → \$8.500 ARS anuales.

Costo Tecnológico Total Inicial: \$8.500 ARS.

21.3 Costos de Operación y Mantenimiento

Analizar qué costos podrían aparecer luego de la puesta en producción del sistema:

- **Renovación anual del dominio: ** **\$8.500 ARS/año**.

- **Mantenimiento correctivo y evolutivo:** Soporte de 4 horas mensuales ante caídas de cuentas de Cloudinary o actualizaciones de seguridad. Costo anual estimado: **\$96.000 ARS**.
- **Upgrade de infraestructura (opcional):** Si el volumen de datos excede los tiers gratuitos, un upgrade a planes pagos básicos de MongoDB Atlas y Cloudinary sumaría aproximadamente **\$50.000 ARS/mes**.

21.4 – Costo total estimados

Concepto de Costo	Tipo de Costo	Frecuencia	Monto (\$ARS)
Recursos Humanos (Equipo Dev)	Desarrollo	Pago Único	\$1.170.000 ARS
Infraestructura Cloud y API	Tecnológico	Mensual	\$0 ARS
Dominio Web (caan.org.ar)	Operativo	Anual	\$8.500 ARS
Soporte y Mantenimiento Técnico	Mantenimiento	Anual	\$96.000 ARS
Costo Total de Desarrollo e Inicio			\$1.178.500 ARS
Costo de Operación Anual Posterior			\$104.500 ARS

Cabe destacar que todos los valores presentados en este análisis de costos son de carácter estimativo y teórico, simulando un escenario profesional de mercado. Al tratarse de un proyecto final de cursada con fines sociales y sin fines de lucro para una ONG, el desarrollo se realiza de forma *adhonorem*. De esta manera, el presupuesto refleja el costo de oportunidad del equipo de trabajo y el valor real del software, permitiendo que la institución destine íntegramente sus recursos económicos a sus fines operativos esenciales.

22 Planificación del Proyecto

22.1 Cronograma General

Sprint 1: Cimientos de la Arquitectura y Persistencia Core

- *Etapas del ciclo de vida involucradas:* **Análisis** (refinamiento de historias de usuario) y **Diseño** (modelado de base de datos).

- *Componentes y Tareas:*
 - Configuración inicial de la estructura del monorepo utilizando *pnpm workspaces*.
 - Definición formal de esquemas de datos, validaciones con Zod e indexación en la base de datos.
 - Establecimiento de la conexión segura hacia el clúster cloud de MongoDB Atlas.
 - **Desarrollo** backend: Implementación del modelo, controladores y rutas CRUD para la gestión de animales.

Sprint 2: Capa de Seguridad e Interfaz del Adoptante

- *Etapas del ciclo de vida involucradas:* **Diseño** (prototipado de componentes visuales) y **Desarrollo** (codificación frontend y seguridad).
- *Componentes y Tareas:*
 - Maquetación del sistema de diseño responsivo utilizando Tailwind CSS v4 bajo enfoque *Mobile-First*.
 - Implementación de la infraestructura de seguridad: Autenticación segura mediante esquema dual de tokens JWT (Access y Refresh tokens encapsulados en cookies httpOnly).
 - Desarrollo en React del catálogo público transaccional, incorporando filtros dinámicos por estados y paginación asíncrona.

Sprint 3: Gestión de Contenidos, Calidad y Lanzamiento

- *Etapas del ciclo de vida involucradas:* **Desarrollo** (módulos administrativos), **Testing** (pruebas integrales) e **Implementación** (puesta en producción).
- *Componentes y Tareas:*
 - Programación algorítmica del módulo de subida multimedia mediante el patrón *Chain of Responsibility* (mecanismo de *fallback* para cuotas de Cloudinary).
 - Desarrollo de la API de noticias e integración del editor enriquecido *BlockNote* en el panel administrativo.
 - Construcción integral del *dashboard* privado para los administradores y voluntarios del refugio.
 - Ejecución de la estrategia de **Testing**: Pruebas de sistema de extremo a extremo (E2E) y validación de criterios de aceptación con la ONG.
 - Fase de **Implementación**: Despliegue continuo automatizado del Frontend en Vercel y el Backend en Render para la puesta en producción definitiva.

22.2 Hitos del Proyecto

- **Hito 1 (Día 5):** Requerimientos validados y Arquitectura de Base de Datos aprobada.
- **Hito 2 (Día 12):** Conexión e infraestructura de la API CRUD de animales validada de forma integral contra MongoDB Atlas.
- **Hito 3 (Día 24):** Sistema de Autenticación de doble token JWT en cookies httpOnly integrado y operativo.
- **Hito 4 (Día 36):** Panel de control administrativo completado (módulos funcionales de animales y noticias integrados con el editor *BlockNote* y carga multimedia).
- **Hito 5 (Día 42):** Portal CAAN desplegado exitosamente en los servidores de producción y pruebas de aceptación aprobadas por la ONG.

22.3 Entregables

Identificar los principales productos o resultados tangibles generados en las fechas clave del cronograma:

- **Entregable 1 (Día 14):** Base de código del Backend estable con API de gestión de animales documentada y script de conexión cloud.
- **Entregable 2 (Día 28):** Código fuente del Frontend con el catálogo público interactivo y el sistema global de sesión de usuarios operativo.
- **Entregable 3 (Día 40):** Dashboard administrativo completamente integrado con el editor *BlockNote* y el sistema de subida asíncrona de imágenes.
- **Entregable 4 (Día 42):** Manual de Usuario Administrador (diseñado en PDF simplificado para el staff de la protectora) y Documento Integral de Gestión del Proyecto.

22.4 Dependencias

- **Dependencia API $\rightarrow$ Frontend:** Las vistas de administración, creación y edición de animales en React dependen críticamente de que la API de subida a Cloudinary (upload.routes.js) y el controlador de lógica (animals.controller.js) estén validados con Zod y funcionales en el backend.
 - *Impacto en la planificación:* Si el desarrollo del backend sufre un retraso, el frontend se ve bloqueado para realizar pruebas con datos reales, postergando los testeos del Sprint 3.
- **Dependencia JWT $\rightarrow$ Rutas Protegidas:** La maquetación de los flujos de seguridad del cliente depende de la correcta configuración y emisión de cabeceras de respuesta HTTP desde Express.

- *Impacto en la planificación:* Cualquier error o mala configuración en las políticas de CORS inter-dominio en el entorno local frena por completo el avance de las llamadas de Axios del Dashboard.

23. Gestión de Cambios

23.1 Escenarios de Cambio

1. **Escenario C1 (Solicitud de Pasarela de Pagos integrada):** La Comisión Directiva del CAAN solicita a mitad del desarrollo que la sección de donaciones no sea estática, sino que permita aportar dinero de forma directa mediante la integración con Mercado Pago.
2. **Escenario C2 (Saturación de cuenta Cloudinary):** Durante las pruebas de carga del panel, la cuenta principal de Cloudinary se suspende repentinamente, deteniendo la carga de imágenes en producción.
3. **Escenario C3 (Incompatibilidad de BlockNote con React 19):** Al integrar el editor de texto, se detectan fallos críticos de dependencias cruzadas no soportadas nativamente por la versión más reciente de React.
4. **Escenario C4 (Reducción del tiempo de entrega):** La cátedra de la universidad adelanta de forma imprevista la fecha final de entrega del trabajo práctico 5 días antes de lo previsto por razones institucionales.
5. **Escenario C5 (Filtros avanzados en catálogo):** Se solicita expandir la lógica de filtros del catálogo de adopciones para incluir parámetros adicionales que originalmente no estaban en el alcance básico (ej. rango de edad o compatibilidad con otros animales).

23.2 Análisis de Impacto

ÍD Cambio	Impacto sobre el Alcance	Impacto sobre el Tiempo	Impacto sobre los Costos	Impacto sobre la Calidad
C1	Alto: Requiere integrar el SDK externo y tablas extras de transacciones.	Alto: Retraso estimado de 7 a 10 días en el sprint activo.	Medio: Costos de comisiones transaccionales de la plataforma.	Bajo: No altera la calidad del núcleo del catálogo de animales.

ÍD Cambio	Impacto sobre el Alcance	Impacto sobre el Tiempo	Impacto sobre los Costos	Impacto sobre la Calidad
C2	Bajo: La funcionalidad ya está desarrollada; es un evento puramente operativo.	Bajo: Se mitiga de forma automática en el código.	Bajo: Las cuentas espejo secundarias configuradas son gratuitas.	Alto: Mantiene el portal disponible previniendo la degradación del servicio.
C3	Bajo: Exige buscar alternativas ágiles (ej. Quill.js) o forzar la instalación.	Medio: Demora de 2 a 3 días por investigación y refactorización.	Bajo: Herramientas de código abierto sin costo de licencias.	Bajo: Entrega el mismo resultado visual de cara al voluntario.
C4	Alto: Obliga de forma drástica a recortar historias de usuario secundarias del Backlog.	Alto: Menor cantidad de días disponibles para el desarrollo activo.	Bajo: No altera los costos teóricos asumidos por el equipo.	Alto: Reduce el margen para realizar pruebas de sistema exhaustivas.
C5	Medio: Modificación de los endpoints de consulta y de la lógica de estados del cliente.	Bajo: Demora estimada de 1 a 2 días de desarrollo.	Bajo: Sin impacto económico en el presupuesto.	Bajo: Eleva la calidad de la experiencia de usuario (UX) para los adoptantes.

23.3 Estrategias de Gestión

Acciones planificadas para procesar los cambios sin comprometer la estabilidad del proyecto:

- **Flujo de Aprobación de Cambios:** Cualquier solicitud de cambio o nueva funcionalidad debe ser evaluada de forma rigurosa por el *Product Owner* (Tomás Sollberger), quien analizará su viabilidad respecto al valor del negocio y el impacto real en los tiempos.
- **Buffer de Tiempo de Contingencia:** El cronograma del Sprint 3 contempla un margen técnico de 3 días dedicados exclusivamente a contingencias; este espacio absorbe de forma orgánica los cambios menores como las correcciones de librerías (C3) o ampliación de filtros (C5).

- **Repriorización del Backlog (Scrum):** Ante la aprobación de un cambio de alto impacto (como Mercado Pago - C1), se negociará la remoción de una funcionalidad del Sprint 3 de menor prioridad (como la maquetación avanzada de noticias), manteniendo fijo el tiempo de entrega y la calidad, pero flexibilizando el alcance.

24. Seguimiento y Control del Proyecto

24.1 Estado Actual del Proyecto

Funcionalidades Finalizadas:

- Configuración inicial de la arquitectura de monorepo (package.json, pnpm-workspace.yaml).
- Conexión a MongoDB Atlas y configuración segura de variables de entorno en el backend.
- Diseño y testeo de esquemas de validación con Zod para usuarios, noticias y animales.
- Modelos de datos y rutas REST base del backend para los módulos de Perros y Noticias

Funcionalidades en Desarrollo:

- Sistema de seguridad de autenticación con doble token JWT y persistencia en cookies httpOnly.
- Interceptor asíncrono de Axios en el frontend para el proceso automático de *Refresh Token*.
- Integración de almacenamiento de medios con Cloudinary (algoritmo de conmutación en proceso).

Funcionalidades Pendientes:

- Dashboard administrativo interno (interfaces visuales CRUD para el staff).
- Vista pública del catálogo interactivo de adopciones con filtros dinámicos en React.
- Integración del editor enriquecido *BlockNote* en los formularios de creación de contenido.

24.2 Indicadores de Avance

Métricas simples y cuantitativas seleccionadas para evaluar el progreso del desarrollo:

Burndown Chart del Sprint: Gráfico diario que mide el trabajo pendiente en relación con el tiempo restante, evaluando la velocidad de resolución de los Story Points del Sprint activo.

Porcentaje de Historias de Usuario Completadas: Calculado matemáticamente de la siguiente manera:

$$\% \text{ Completitud} = \left(\frac{\text{Story Points Terminado y Probado (Done)}}{\text{Total Story Points del Backlog (63 SP)}} \right) \times$$

Velocidad del Equipo: Cantidad promedio de Story Points que el equipo de tres personas logra certificar como terminados al finalizar una iteración de 2 semanas (Velocidad histórica estimada: 21 SP por Sprint).

24.3 Riesgos Actuales

Análisis de vigencia de los factores de riesgo del proyecto:

- **Riesgos Vigentes:** El riesgo de saturación de cuota en Cloudinary (R1) y el riesgo humano de indisponibilidad por sobrecarga (R2) se mantienen como los más latentes debido a la inminente integración del panel administrativo masivo. La correcta implementación de la arquitectura de la capa de servicios (*Service Layer*) y el middleware global son urgentes para blindar al equipo contra bloqueos de desarrollo.
- **Nuevos Riesgos Identificados:** Se ha detectado que las restricciones de seguridad recientes en navegadores basados en Chromium bloquean las cookies httpOnly en entornos locales si no se operan bajo protocolos HTTPS seguros, lo que podría obligar al equipo a configurar un proxy inverso local para el testing del Sprint 2.

24.4 Acciones Correctivas

Medidas de contingencia preestablecidas ante desviaciones del plan original:

- **Ante retrasos en Sprints (Desvío de Tiempo):** El Scrum Master convocará a una reunión extraordinaria de replanificación para recortar el alcance de componentes no críticos (por ejemplo, reducir el diseño interactivo de las FAQ a un acordeón nativo plano en Tailwind sin animaciones complejas), reenfocando el esfuerzo humano en cerrar el CRUD core y la autenticación.
- **Ante fallas de calidad de código (Bugs Recurrentes):** Se aplicará un congelamiento temporal de características (*Feature Freeze*), deteniendo el desarrollo de funciones nuevas y dedicando las primeras jornadas de la siguiente iteración exclusivamente a saldar la deuda técnica y robustecer las validaciones de Zod.

25. Conclusión

El desarrollo de este documento integral de gestión y planificación para el portal web del CAAN ha puesto de manifiesto la enorme utilidad práctica de aplicar las técnicas y marcos metodológicos de la ingeniería de software en escenarios de la vida real.

La descomposición sistemática del alcance y la estimación basada en *Story Points* a través de dinámicas de *Planning Poker* permitieron aterrizar las expectativas del equipo bajo métricas realistas. Esto nos dio la capacidad de asignar valores de esfuerzo basados en la complejidad arquitectónica y la incertidumbre en lugar de basarnos en meras intuiciones temporales.

Asimismo, el análisis detallado de costos evidenció que, aun operando bajo infraestructuras en la nube y API totalmente gratuitas (*tiers sandbox*), existen costos humanos operativos implícitos muy altos que deben cuantificarse de cara a justificar de forma formal la viabilidad y la sostenibilidad del software ante una organización.

Por otro lado, la planificación estructurada sobre hitos y dependencias cruzadas preparó al equipo para reaccionar de forma metódica ante imprevistos técnicos, utilizando pautas formales de gestión de cambios y métricas de avance específicas como el control de velocidad. En definitiva, este trabajo integrador consolida la enseñanza de que un producto de software robusto no surge únicamente del acto aislado de escribir líneas de código, sino de un proceso disciplinado y holístico de gestión que asegure la calidad del producto, la previsibilidad del cronograma y el impacto real sobre los usuarios finales.